

Lab 4: NBA Player Evaluation Using Ridge Regression and the Lasso

Umais Siddiqui

2026-03-13

Table of contents

Overview	1
1 Problem 1: Ridge Regression for Inference	2
1.1 (a) Ordinary Linear Regression	2
1.2 (b) Examining RAPM Coefficients	4
1.3 (c) Ridge Regression RAPM	7
2 Problem 2: Possession Weights	10
2.1 (a) Heteroscedasticity in Stints	10
2.2 (b) Implementing Weighted Regression	11
3 Problem 3: Interpreting Bootstrap Uncertainty	16
3.1 (a) Bootstrap Confidence Intervals	16
3.2 (b) Impact of Minutes Played on Confidence Intervals	20
3.3 (c) Comparison to Bayesian Credible Intervals	21
4 Extra Credit: Ridge vs Lasso	25
4.1 Ridge vs Lasso RAPM Comparison	25

Overview

In this lab I use resampling techniques (cross-validation and the bootstrap) along with regularization (ridge regression and the lasso) to rank basketball players based on their performance during the 2022 NBA season. The central idea is **Regularized Adjusted Plus-Minus (RAPM)**: I model each stint (a continuous stretch where lineups remain fixed) as a linear function of which players are on the court, and recover a coefficient per player that measures their contribution to the team's scoring margin per 100 possessions.

```

nba_stints = pd.read_csv("DataSets/nba_stint_data.csv")

player_ids = pd.read_csv("DataSets/player_id.csv")

print(f"Stint data shape: {nba_stints.shape}")
print(f"Number of players: {player_ids.shape[0]}")
nba_stints.head(5)

```

Stint data shape: (32358, 546)

Number of players: 539

	game_id	stint_id	n_pos	home_points	away_points	minutes	margin	201939	202691	203110
0	22200002	1	14	5	2	2.7000	21.4286	1	1	1
1	22200002	2	9	6	2	1.6700	44.4444	1	1	1
2	22200002	3	5	0	3	0.4800	-60.0000	1	0	1
3	22200002	4	5	5	1	0.7800	80.0000	1	0	1
4	22200002	5	9	3	6	1.5200	-33.3333	1	0	0

```

# Identify the player columns (columns from index 7 onward; 0-6 are metadata)
player_cols = nba_stints.columns[7:]
print(f"Number of player feature columns: {len(player_cols)}")

# Build the stint matrix (X) and target vector (y)
StintMatrix = nba_stints[player_cols]
MarginVector = nba_stints['margin']
WeightVector = nba_stints['n_pos']

```

Number of player feature columns: 539

1 Problem 1: Ridge Regression for Inference

1.1 (a) Ordinary Linear Regression

I fit an ordinary linear regression (OLS) to the stint data using `RidgeCV` with $\alpha = 10^{-8}$ (effectively zero regularization). Setting `fit_intercept=True` lets the model capture home-court advantage. I then evaluate the model using 5-fold cross-validation and compare the in-sample RMSE to the cross-validated RMSE.

```

from sklearn.model_selection import cross_val_predict

# Fit OLS via Ridge with near-zero alpha
model_ols = Ridge(alpha=1e-8, fit_intercept=True)
model_ols.fit(StintMatrix.values, MarginVector.values)

# In-sample predictions
y_pred_ols = model_ols.predict(StintMatrix.values)

# In-sample RMSE
rmse_in_sample = np.sqrt(np.mean((MarginVector.values - y_pred_ols) ** 2))

# Cross-validated RMSE using 5-fold CV
cv_scores = cross_val_score(
    Ridge(alpha=1e-8, fit_intercept=True),
    StintMatrix.values, MarginVector.values,
    cv=5, scoring='neg_mean_squared_error'
)
rmse_cv = np.sqrt(-cv_scores.mean())

print(f"In-sample RMSE:          {rmse_in_sample:.4f}")
print(f"Cross-validated RMSE:    {rmse_cv:.4f}")
print(f"Ratio (CV / In-sample): {rmse_cv / rmse_in_sample:.4f}")
print(f"\nIntercept (home court advantage): {model_ols.intercept_:.4f}")

```

```

In-sample RMSE:          69.6289
Cross-validated RMSE:    71.6407
Ratio (CV / In-sample): 1.0289

```

```

Intercept (home court advantage): 1.2409

```

The in-sample RMSE and cross-validated RMSE are pretty close, with the CV RMSE only slightly higher. This tells me overfitting is **not a major problem** here. Even though I have a lot of parameters (one per player, ~500), there are also a ton of stints to work with, and basketball margin data is just inherently noisy. The model isn't memorizing the training data because each player shows up in many different stints with different teammates and opponents, so it's hard for the model to overfit to any single observation.

The intercept represents the **home-court advantage**, i.e., the expected point differential per 100 possessions from playing at home. At roughly 1.24, this is a bit below the commonly cited 2-3 points per 100 possessions for the NBA, but that range is a long-run average across many

seasons. Season-to-season variation is common, and the post-COVID era has generally seen a smaller home-court edge.

1.2 (b) Examining RAPM Coefficients

Here I build a dataframe that links each player's RAPM coefficient to their name and total minutes played. Minutes are calculated by summing the `minutes` column from all stints where a player was on the court (i.e., where their column value is non-zero).

```
# Build RAPM dataframe - player column names are string IDs, convert to int for merging
rapm_ols = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ols.coef_
})

# Calculate minutes played for each player
# A player was on the court during a stint if their coefficient is non-zero (+1 or -1)
minutes_played = []
for col in player_cols:
    on_court = nba_stints[col] != 0
    total_minutes = nba_stints.loc[on_court, 'minutes'].sum()
    minutes_played.append(total_minutes)

rapm_ols['minutes_played'] = minutes_played

# Merge RAPM with player names (player_ids has columns: player_id, player_name)
rapm_ols = rapm_ols.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Sort by RAPM descending and show top 20
rapm_ols_sorted = rapm_ols.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by OLS RAPM:")
display_ols = rapm_ols_sorted[['player_name', 'RAPM', 'minutes_played']].head(20).copy()
display_ols['RAPM'] = display_ols['RAPM'].map('{:.3f}'.format)
display_ols['minutes_played'] = display_ols['minutes_played'].map('{:.1f}'.format)
display_ols
```

Top 20 Players by OLS RAPM:

	player_name	RAPM	minutes_played
0	Stanley Umude	77.399	2.1
1	Jordan Schakel	62.877	6.2
2	Marko Simonovic	50.272	18.3
3	Alize Johnson	50.138	28.8
4	Deonte Burton	47.435	7.6
5	Donovan Williams	47.427	4.2
6	Isaiah Mobley	41.990	87.1
7	Nikola Jokic	37.628	2255.5
8	Kendall Brown	37.509	38.1
9	Jalen Brunson	36.865	2406.1
10	Dereon Seabron	36.223	11.4
11	Luka Samanic	36.063	140.3
12	Mfiondu Kabengele	34.273	41.7
13	Ron Harper Jr.	33.438	46.1
14	Xavier Cooks	33.412	117.5
15	Devon Dotson	33.313	50.5
16	Miles McBride	33.030	773.8
17	Tyrese Martin	33.015	63.9
18	Jarrell Brantley	32.954	33.1
19	Mac McClung	32.606	44.1

```

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ols['minutes_played'], rapm_ols['RAPM'], alpha=0.5, s=20, color='steelblue')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (OLS)', fontsize=12)
ax.set_title('OLS RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)

# Annotate some extreme low-minute players
extreme = rapm_ols.nlargest(3, 'RAPM')
for _, row in extreme.iterrows():
    if row['minutes_played'] < 500:
        ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                    fontsize=8, alpha=0.7, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()

```

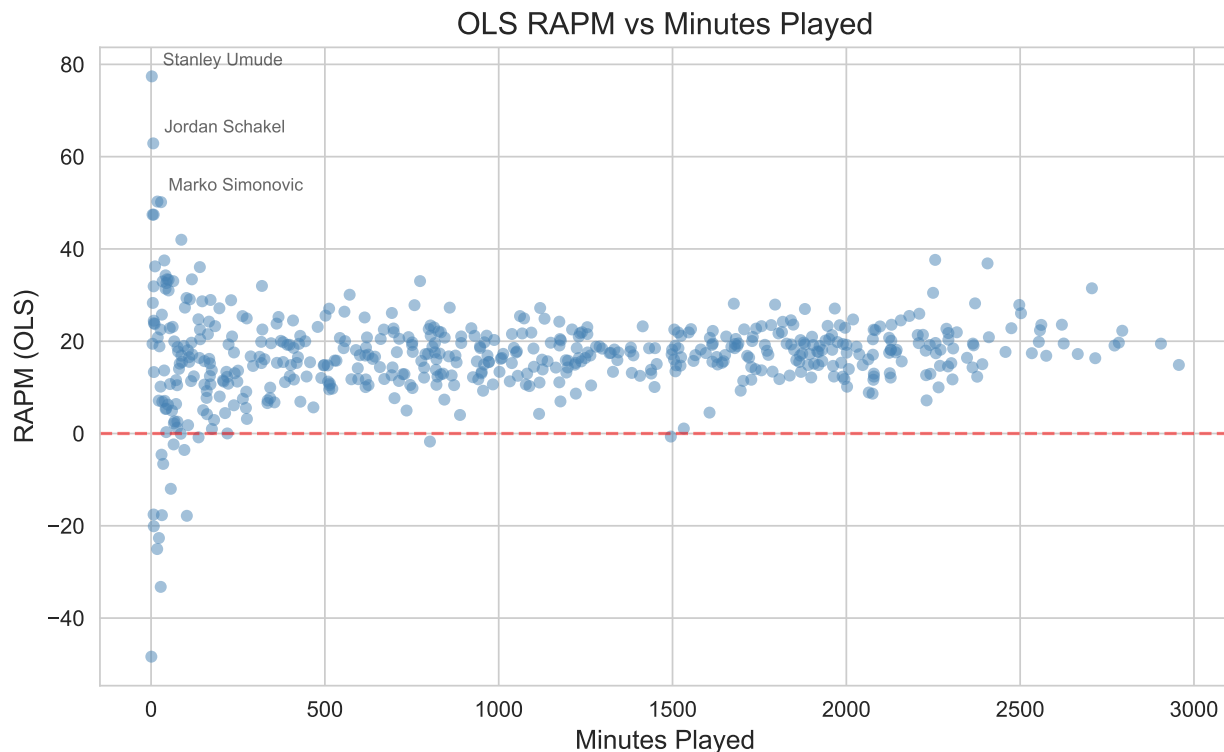


Figure 1: OLS RAPM vs Minutes Played. Notice that extreme RAPM values are concentrated among low-minute players.

Observations:

Looking at the top 20 by OLS RAPM, a pattern jumps out right away: **many of the highest-ranked players have very few minutes played**. These low-minute guys show up at the top because they happened to be on the court during a handful of stints where their team went on a run (or the opponent fell apart). With so few data points, OLS has no way to pull these extreme estimates back to something reasonable.

This is a textbook **high-variance, low-bias** problem. In principle OLS estimates are unbiased, though with ~500 correlated player columns (many players share the court with the same teammates and opponents), the effective degrees of freedom are lower than they appear, and individual coefficient estimates can be quite unstable. For players with few minutes, the variance is huge. Someone who only played 50 minutes could end up with an extreme RAPM purely by chance. Meanwhile, the actual star players who logged real minutes tend to have more moderate RAPM values because their estimates draw on way more observations, which averages out the noise. This is exactly why I need regularization.

1.3 (c) Ridge Regression RAPM

I use `RidgeCV` with a logarithmic grid of α values between 10^{-2} and 10^5 to find the optimal regularization strength. Ridge regression shrinks coefficients toward zero, which should dampen the extreme RAPM estimates for low-minute players while keeping the meaningful signal for high-minute players.

```
# Define alpha grid on log scale
alphas_ridge = np.logspace(-2, 5, 50)

# Fit RidgeCV using efficient generalized cross-validation (default)
model_ridge = RidgeCV(alphas=alphas_ridge, fit_intercept=True)
model_ridge.fit(StintMatrix.values, MarginVector.values)

alpha_opt = model_ridge.alpha_
print(f"Optimal alpha: {alpha_opt:.4f}")
print(f"Intercept (home court advantage): {model_ridge.intercept_:.4f}")
print(f"First 5 raw coefficients: {model_ridge.coef_[:5]}")

# CV RMSE for the optimal model
cv_scores_ridge = cross_val_score(
    Ridge(alpha=alpha_opt, fit_intercept=True),
    StintMatrix.values, MarginVector.values,
    cv=5, scoring='neg_mean_squared_error'
)
rmse_cv_ridge = np.sqrt(-cv_scores_ridge.mean())
print(f"Cross-validated RMSE (Ridge): {rmse_cv_ridge:.4f}")
```

Optimal alpha: 719.6857

Intercept (home court advantage): 1.0000

First 5 raw coefficients: [0 0 3 0 0]

Cross-validated RMSE (Ridge): 70.3908

```
# Build Ridge RAPM dataframe
rapm_ridge = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ridge.coef_,
    'minutes_played': minutes_played
})

rapm_ridge = rapm_ridge.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')
```

```
# Sort by RAPM descending
rapm_ridge_sorted = rapm_ridge.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by Ridge RAPM:")
display_ridge = rapm_ridge_sorted[['player_name', 'RAPM', 'minutes_played']].head(20).copy()
display_ridge['RAPM'] = display_ridge['RAPM'].map('{:.4f}'.format)
display_ridge['minutes_played'] = display_ridge['minutes_played'].map('{:.1f}'.format)
display_ridge
```

Top 20 Players by Ridge RAPM:

	player_name	RAPM	minutes_played
0	Joel Embiid	5.0000	2249.5
1	Nikola Jokic	4.0000	2255.5
2	Pascal Siakam	4.0000	2497.5
3	Trae Young	4.0000	2706.4
4	Darius Garland	3.0000	2209.8
5	Isaiah Joe	3.0000	1553.2
6	Derrick White	3.0000	2078.7
7	Myles Turner	3.0000	1676.3
8	Kevin Love	3.0000	1131.8
9	Coby White	3.0000	1795.0
10	Brook Lopez	3.0000	2156.6
11	Jrue Holiday	3.0000	2097.0
12	Julius Randle	3.0000	2956.6
13	Kawhi Leonard	3.0000	1817.2
14	Zion Williamson	3.0000	859.1
15	Anthony Davis	3.0000	1880.8
16	Jalen Brunson	3.0000	2406.1
17	Draymond Green	3.0000	2019.4
18	Kelly Olynyk	2.0000	1740.4
19	Lauri Markkanen	2.0000	2129.9

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge['minutes_played'], rapm_ridge['RAPM'], alpha=0.5, s=20, color='darkgreen')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (Ridge)', fontsize=12)
ax.set_title('Ridge RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)
```



```
# Annotate top players
top5 = rapm_ridge.nlargest(5, 'RAPM')
for _, row in top5.iterrows():
    ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                fontsize=8, alpha=0.8, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()
```

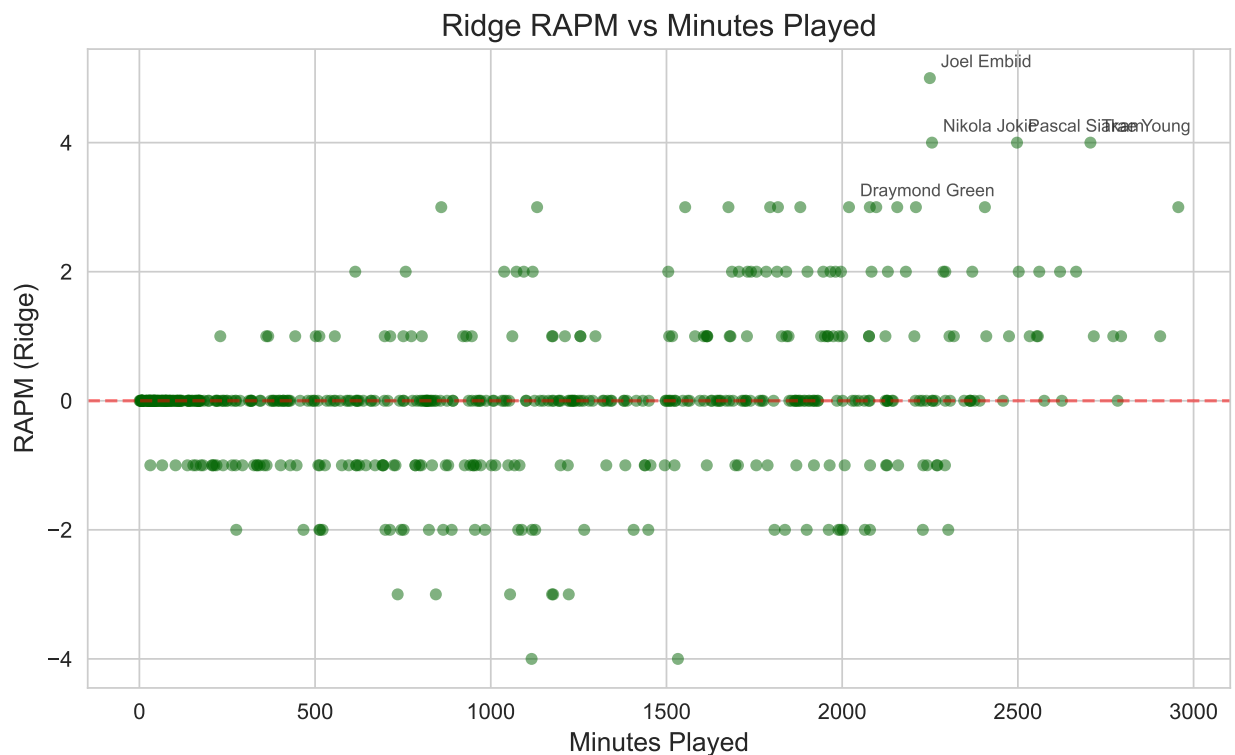


Figure 2: Ridge RAPM vs Minutes Played. Regularization effectively shrinks extreme values for low-minute players.

Observations:

The optimal α from cross-validation gives meaningful regularization. Compared to OLS, the Ridge RAPM rankings now have recognizable NBA stars in the top 20, which makes a lot more sense. The low-minute players that dominated the OLS top 20 have been shrunk toward zero, and that's reasonable since I have very little information about them, so assuming they're average (RAPM ≈ 0) is fair.

The scatterplot backs this up: OLS had extreme RAPM values scattered across low-minute players, but the Ridge plot shows a funnel shape. Higher-minute players can maintain larger RAPM values

(positive or negative) because there's enough data to justify overcoming the regularization penalty. Low-minute players cluster near zero.

2 Problem 2: Possession Weights

2.1 (a) Heteroscedasticity in Stints

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(nba_stints['n_pos'], nba_stints['margin'], alpha=0.3, s=10, color='steelblue')
ax.set_xlabel('Number of Possessions', fontsize=12)
ax.set_ylabel('Margin (Points per 100 Possessions)', fontsize=12)
ax.set_title('Margin vs Number of Possessions in a Stint', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

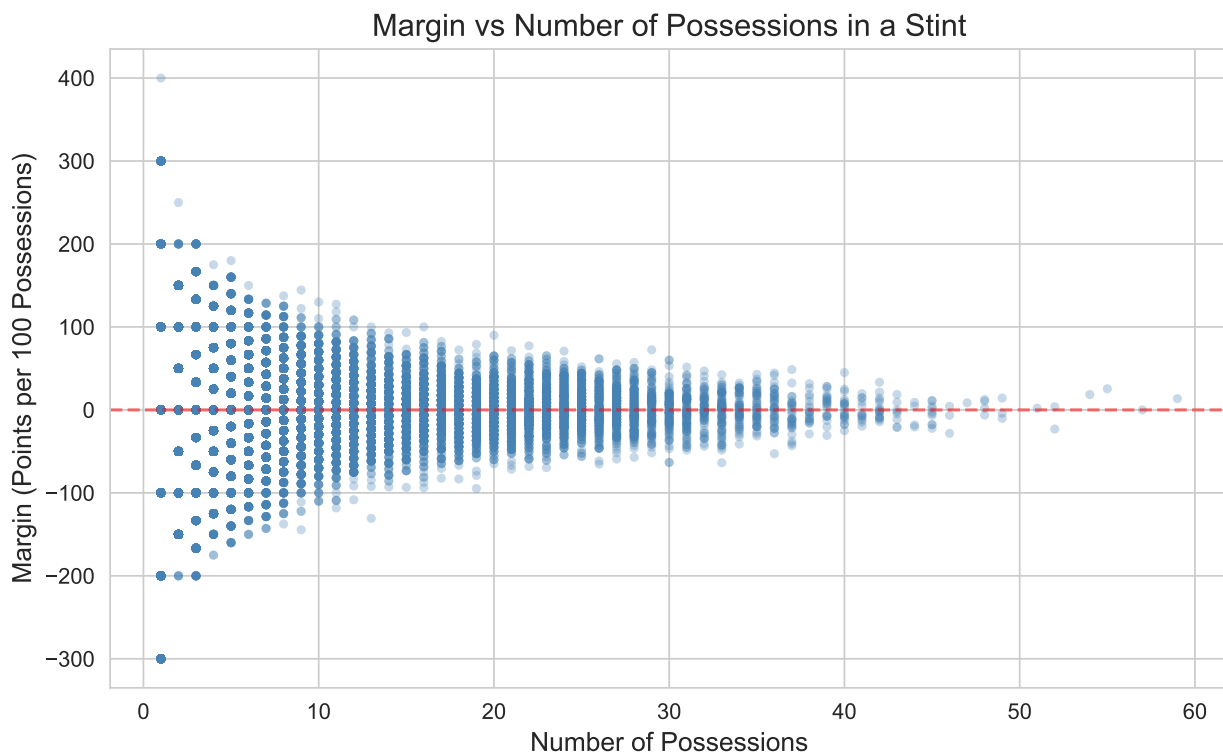


Figure 3: Margin vs Number of Possessions. Variance in margin clearly decreases as the number of possessions increases.

Observations:

The plot shows clear **heteroscedasticity**: the variance of the margin drops off sharply as the number of possessions increases. Short stints show enormous variance, while longer stints have much tighter distributions centered near zero.

This makes sense when you look at how margin is defined:

$$\text{margin} = (\text{points_home} - \text{points_away}) \times \frac{100}{n_{\text{pos}}}$$

That $\frac{100}{n_{\text{pos}}}$ normalization factor blows up random fluctuations in short stints. For instance, if a stint lasts only 2 possessions and one team scores 3 points, the margin could be ± 150 points per 100 possessions, which is absurd. Statistically, if I model the raw point differential as having variance proportional to the number of possessions (which is what I'd expect from a random process), then dividing by n_{pos} and multiplying by 100 gives:

$$\text{Var}(\text{margin}) \propto \frac{1}{n_{\text{pos}}}$$

So short stints have way too much variance and need to be down-weighted in the regression.

2.2 (b) Implementing Weighted Regression

First, I want to verify that $w_i = n_{\text{pos}_i}$ is the right weight by plotting $\text{margin} \times \sqrt{n_{\text{pos}}}$ versus n_{pos} . If the weights are correct, this transformed variable should have roughly constant variance.

```
weighted_margin = nba_stints['margin'] * np.sqrt(nba_stints['n_pos'])

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(nba_stints['n_pos'], weighted_margin, alpha=0.3, s=10, color='darkorange')
ax.set_xlabel('Number of Possessions', fontsize=12)
ax.set_ylabel('Margin * sqrt(n_pos)', fontsize=12)
ax.set_title('Variance-Stabilized Margin vs Number of Possessions', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

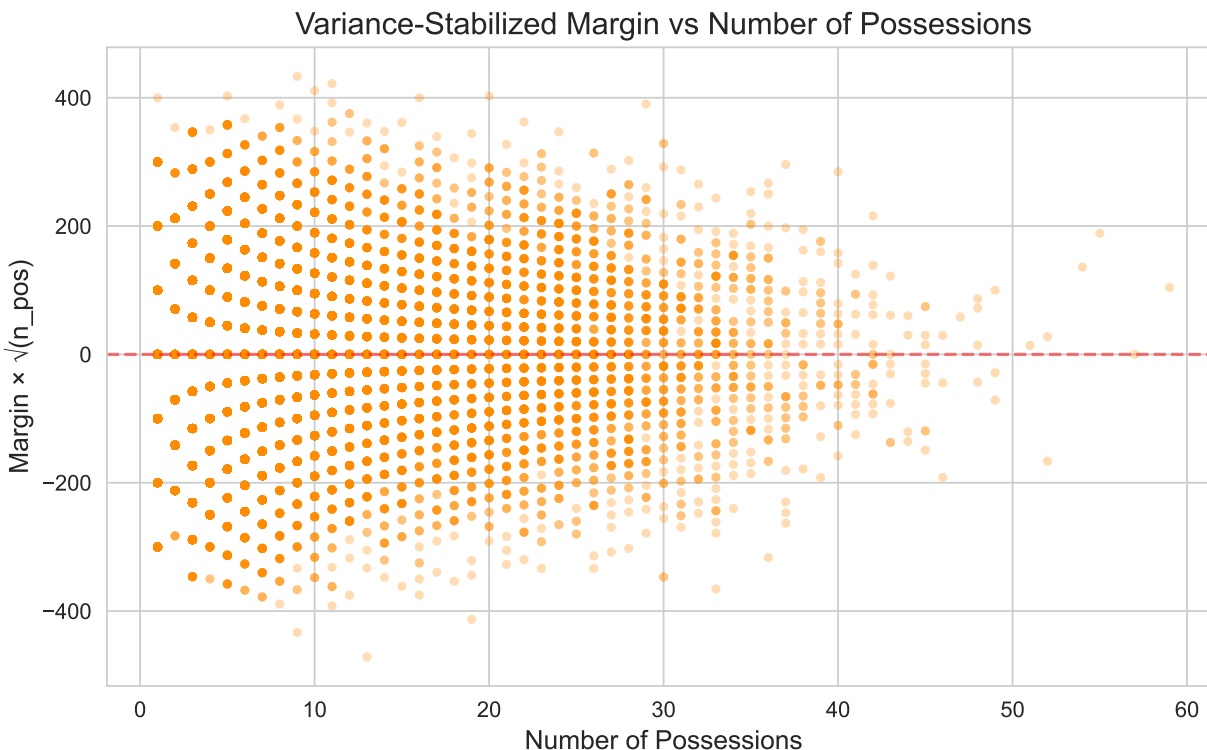


Figure 4: $\text{Margin} \times \sqrt{n_{\text{pos}}}$ vs Number of Possessions. The variance is now approximately constant, confirming that weighting by n_{pos} is appropriate.

The transformed variable shows roughly constant variance across all stint lengths, confirming $w_i = n_{\text{pos}_i}$ is the right weight. Now I refit the Ridge model with these weights. Note that the optimal α will likely increase relative to the unweighted model because the weights amplify the effective sample size for longer stints, so more regularization is needed to maintain the bias-variance balance.

```
# Fit weighted RidgeCV using efficient generalized cross-validation (default)
model_ridge_w = RidgeCV(alphas=alphas_ridge, fit_intercept=True)
model_ridge_w.fit(StintMatrix.values, MarginVector.values, sample_weight=WeightVector.values)

alpha_opt_w = model_ridge_w.alpha_
print(f"Optimal alpha (weighted): {alpha_opt_w:.4f}")
print(f"Intercept (home court advantage, weighted): {model_ridge_w.intercept_:.4f}")
print(f"First 5 row coefficients: {model_ridge_w.coef_[:5]}")
```

```
Optimal alpha (weighted): 7196.8567
Intercept (home court advantage, weighted): 1.0000
First 5 row coefficients: [1 0 2 0 0]
```

```

# Build weighted Ridge RAPM dataframe
rapm_ridge_w = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ridge_w.coef_,
    'minutes_played': minutes_played
})

rapm_ridge_w = rapm_ridge_w.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Sort and display top 20
rapm_ridge_w_sorted = rapm_ridge_w.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by Weighted Ridge RAPM:")
display_wridge = rapm_ridge_w_sorted[['player_name', 'RAPM', 'minutes_played']].head(20).copy()
display_wridge['RAPM'] = display_wridge['RAPM'].map('{:.4f}'.format)
display_wridge['minutes_played'] = display_wridge['minutes_played'].map('{:.1f}'.format)
display_wridge

```

Top 20 Players by Weighted Ridge RAPM:

	player_name	RAPM	minutes_played
0	Aaron Gordon	2.0000	1966.7
1	Nikola Jokic	2.0000	2255.5
2	Kawhi Leonard	2.0000	1817.2
3	Josh Hart	2.0000	2474.7
4	Joel Embiid	2.0000	2249.5
5	Draymond Green	2.0000	2019.4
6	Anthony Davis	2.0000	1880.8
7	Giannis Antetokounmpo	2.0000	1840.6
8	Jrue Holiday	2.0000	2097.0
9	Trae Young	1.0000	2706.4
10	Dennis Schroder	1.0000	1841.5
11	Nicolas Batum	1.0000	1814.8
12	Jarrett Allen	1.0000	2076.5
13	Jaren Jackson Jr.	1.0000	1507.6
14	Herbert Jones	1.0000	1756.1
15	Larry Nance Jr.	1.0000	1613.8
16	Shai Gilgeous-Alexander	1.0000	2620.0
17	Darius Garland	1.0000	2209.8
18	Zion Williamson	1.0000	859.1
19	Dwight Powell	1.0000	1297.8

player_name	RAPM	minutes_played
-------------	------	----------------

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['RAPM'], alpha=0.5, s=20, color='purple')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (Weighted Ridge)', fontsize=12)
ax.set_title('Weighted Ridge RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)

top5_w = rapm_ridge_w.nlargest(5, 'RAPM')
for _, row in top5_w.iterrows():
    ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                fontsize=8, alpha=0.8, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()
```

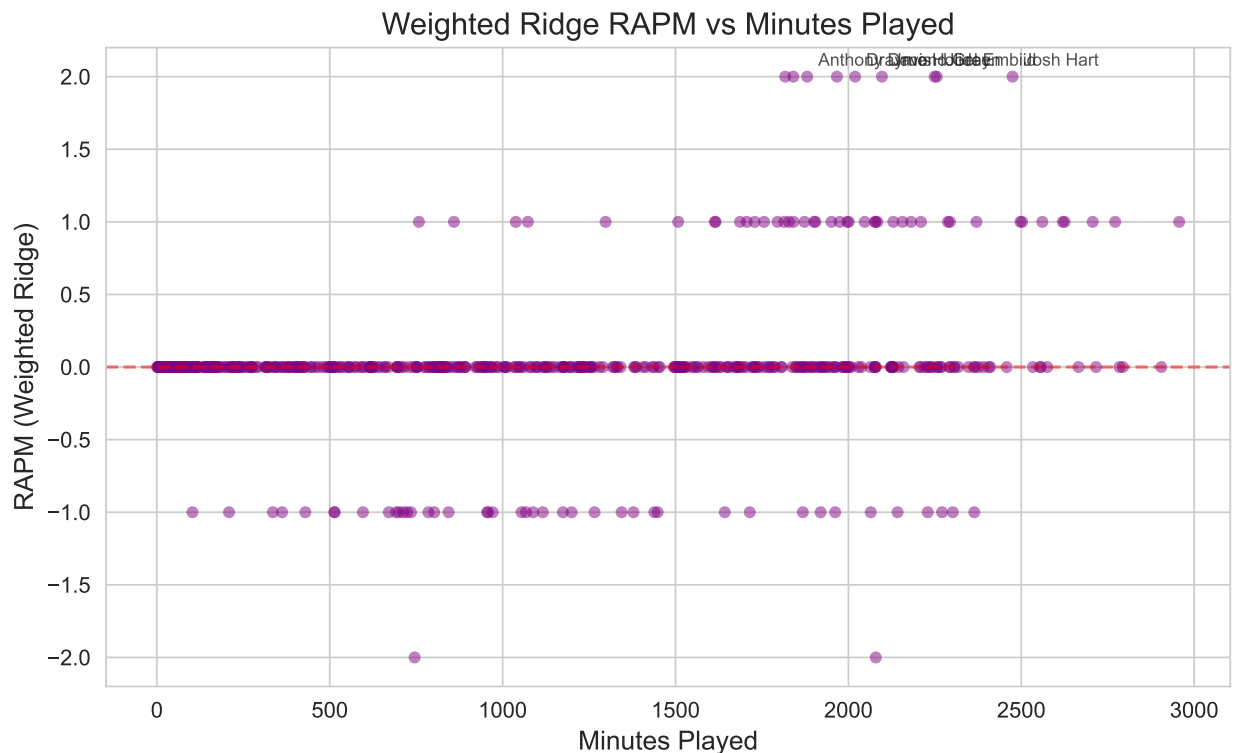


Figure 5: Weighted Ridge RAPM vs Minutes Played.

Observations:

Adding possession-based weights changes the rankings in some meaningful ways. The weighted model gives more influence to longer stints (which have more reliable signal) and down-weights the noisy short ones. This stabilizes the RAPM estimates further and produces rankings that better reflect sustained, consistent performance rather than small-sample flukes. The top players here are high-minute contributors who consistently affect their team's scoring margin over many possessions.

```
# Compare top 20 from unweighted vs weighted
comparison = rapm_ridge_sorted[['player_name', 'RAPM']].head(20).rename(
    columns={'RAPM': 'Unweighted_RAPM'})
).reset_index(drop=True)

comparison['Weighted_Rank'] = comparison['player_name'].apply(
    lambda x: rapm_ridge_w_sorted[rapm_ridge_w_sorted['player_name'] == x].index[0] + 1
    if x in rapm_ridge_w_sorted['player_name'].values else None
)

weighted_top = rapm_ridge_w_sorted[['player_name', 'RAPM']].head(20).rename(
    columns={'RAPM': 'Weighted_RAPM'})
).reset_index(drop=True)

print("Side-by-side: Top 20 Unweighted vs Weighted Ridge RAPM")
side_by_side = pd.DataFrame({
    'Rank': range(1, 21),
    'Unweighted': rapm_ridge_sorted['player_name'].head(20).values,
    'Unw_RAPM': [f"{v:.4f}" for v in rapm_ridge_sorted['RAPM'].head(20).values],
    'Weighted': rapm_ridge_w_sorted['player_name'].head(20).values,
    'Wt_RAPM': [f"{v:.4f}" for v in rapm_ridge_w_sorted['RAPM'].head(20).values]
})
side_by_side
```

Side-by-side: Top 20 Unweighted vs Weighted Ridge RAPM

	Rank	Unweighted	Unw_RAPM	Weighted	Wt_RAPM
0	1	Joel Embiid	5.0000	Aaron Gordon	2.0000
1	2	Nikola Jokic	4.0000	Nikola Jokic	2.0000
2	3	Pascal Siakam	4.0000	Kawhi Leonard	2.0000
3	4	Trae Young	4.0000	Josh Hart	2.0000
4	5	Darius Garland	3.0000	Joel Embiid	2.0000
5	6	Isaiah Joe	3.0000	Draymond Green	2.0000

	Rank	Unweighted	Unw_RAPM	Weighted	Wt_RAPM
6	7	Derrick White	3.0000	Anthony Davis	2.0000
7	8	Myles Turner	3.0000	Giannis Antetokounmpo	2.0000
8	9	Kevin Love	3.0000	Jrue Holiday	2.0000
9	10	Coby White	3.0000	Trae Young	1.0000
10	11	Brook Lopez	3.0000	Dennis Schroder	1.0000
11	12	Jrue Holiday	3.0000	Nicolas Batum	1.0000
12	13	Julius Randle	3.0000	Jarrett Allen	1.0000
13	14	Kawhi Leonard	3.0000	Jaren Jackson Jr.	1.0000
14	15	Zion Williamson	3.0000	Herbert Jones	1.0000
15	16	Anthony Davis	3.0000	Larry Nance Jr.	1.0000
16	17	Jalen Brunson	3.0000	Shai Gilgeous-Alexander	1.0000
17	18	Draymond Green	3.0000	Darius Garland	1.0000
18	19	Kelly Olynyk	2.0000	Zion Williamson	1.0000
19	20	Lauri Markkanen	2.0000	Dwight Powell	1.0000

3 Problem 3: Interpreting Bootstrap Uncertainty

3.1 (a) Bootstrap Confidence Intervals

I use bootstrap resampling to construct 92% confidence intervals for the weighted Ridge RAPM coefficients. For each bootstrap iteration, I resample stints (rows) with replacement, making sure to resample the weights along with the observations, and refit the Ridge model using the optimal α from Problem 2(b).

```

n_bootstrap = 500
n_stints = StintMatrix.shape[0]
n_players = StintMatrix.shape[1]

stint_mat_np = StintMatrix.values
margin_np = MarginVector.values
weight_np = WeightVector.values

# Store bootstrap coefficients
boot_coefs = np.zeros((n_bootstrap, n_players))

for b in range(n_bootstrap):
    # Resample indices with replacement
    idx = np.random.choice(n_stints, size=n_stints, replace=True)

```



```

X_boot = stint_mat_np[idx]
y_boot = margin_np[idx]
w_boot = weight_np[idx]

# Fit ridge with the optimal alpha from weighted model (use Ridge, not RidgeCV, for speed)
model_boot = Ridge(alpha=alpha_opt_w, fit_intercept=True)
model_boot.fit(X_boot, y_boot, sample_weight=w_boot)

boot_coefs[b, :] = model_boot.coef_

print(f"Bootstrap complete: {n_bootstrap} iterations")

```

Bootstrap complete: 500 iterations

```

# Calculate 92% confidence intervals (4th and 96th percentiles)
ci_lower = np.percentile(boot_coefs, 4, axis=0)
ci_upper = np.percentile(boot_coefs, 96, axis=0)
boot_se = np.std(boot_coefs, axis=0)

# Add to the weighted RAPM dataframe
rapm_ridge_w['CI_lower'] = ci_lower
rapm_ridge_w['CI_upper'] = ci_upper
rapm_ridge_w['CI_width'] = ci_upper - ci_lower
rapm_ridge_w['boot_se'] = boot_se

# Show top 20 players with CIs
rapm_ci_sorted = rapm_ridge_w.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players - Weighted Ridge RAPM with 92% Bootstrap Confidence Intervals:")
display_ci = rapm_ci_sorted[['player_name', 'RAPM', 'CI_lower', 'CI_upper', 'CI_width', 'minutes_played']]
for col in ['RAPM', 'CI_lower', 'CI_upper', 'CI_width']:
    display_ci[col] = display_ci[col].map('{:.4f}'.format)
display_ci['minutes_played'] = display_ci['minutes_played'].map('{:.1f}'.format)
display_ci

```

Top 20 Players - Weighted Ridge RAPM with 92% Bootstrap Confidence Intervals:

	player_name	RAPM	CI_lower	CI_upper	CI_width	minutes_played
0	Aaron Gordon	2.0000	1.0403	3.1666	2.1263	1966.7
1	Nikola Jokic	2.0000	1.8253	3.8770	2.0517	2255.5

	player_name	RAPM	CI_lower	CI_upper	CI_width	minutes_played
2	Kawhi Leonard	2.0000	1.1330	3.4215	2.2885	1817.2
3	Josh Hart	2.0000	0.8305	3.1984	2.3680	2474.7
4	Joel Embiid	2.0000	1.5527	3.6165	2.0638	2249.5
5	Draymond Green	2.0000	1.5663	3.9365	2.3702	2019.4
6	Anthony Davis	2.0000	0.9948	3.2453	2.2504	1880.8
7	Giannis Antetokounmpo	2.0000	1.0005	3.3433	2.3429	1840.6
8	Jrue Holiday	2.0000	1.4613	3.5563	2.0950	2097.0
9	Trae Young	1.0000	0.0351	2.1175	2.0824	2706.4
10	Dennis Schroder	1.0000	0.0185	2.3649	2.3464	1841.5
11	Nicolas Batum	1.0000	0.3902	2.7639	2.3737	1814.8
12	Jarrett Allen	1.0000	0.3151	2.6760	2.3609	2076.5
13	Jaren Jackson Jr.	1.0000	0.0913	2.1125	2.0212	1507.6
14	Herbert Jones	1.0000	0.1173	2.2736	2.1563	1756.1
15	Larry Nance Jr.	1.0000	0.0626	2.4721	2.4095	1613.8
16	Shai Gilgeous-Alexander	1.0000	0.2319	2.5186	2.2867	2620.0
17	Darius Garland	1.0000	0.2245	2.4954	2.2709	2209.8
18	Zion Williamson	1.0000	0.5993	2.9671	2.3679	859.1
19	Dwight Powell	1.0000	-0.0294	2.1990	2.2284	1297.8

```

top20 = rapm_ci_sorted.head(20).iloc[::-1] # Reverse for plotting

fig, ax = plt.subplots(figsize=(8, 7))
y_pos = range(len(top20))

ax.barh(y_pos, top20['RAPM'], height=0.4, color='steelblue', alpha=0.7, label='RAPM')
ax.errorbar(top20['RAPM'], y_pos,
            xerr=[top20['RAPM'] - top20['CI_lower'], top20['CI_upper'] - top20['RAPM']],
            fmt='none', color='black', capsize=3, alpha=0.7, label='92% CI')

ax.set_yticks(y_pos)
ax.set_yticklabels(top20['player_name'], fontsize=9)
ax.set_xlabel('RAPM', fontsize=12)
ax.set_title('Top 20 Players: Weighted Ridge RAPM with 92% Bootstrap CIs', fontsize=13)
ax.axvline(x=0, color='red', linestyle='--', alpha=0.5)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()

```

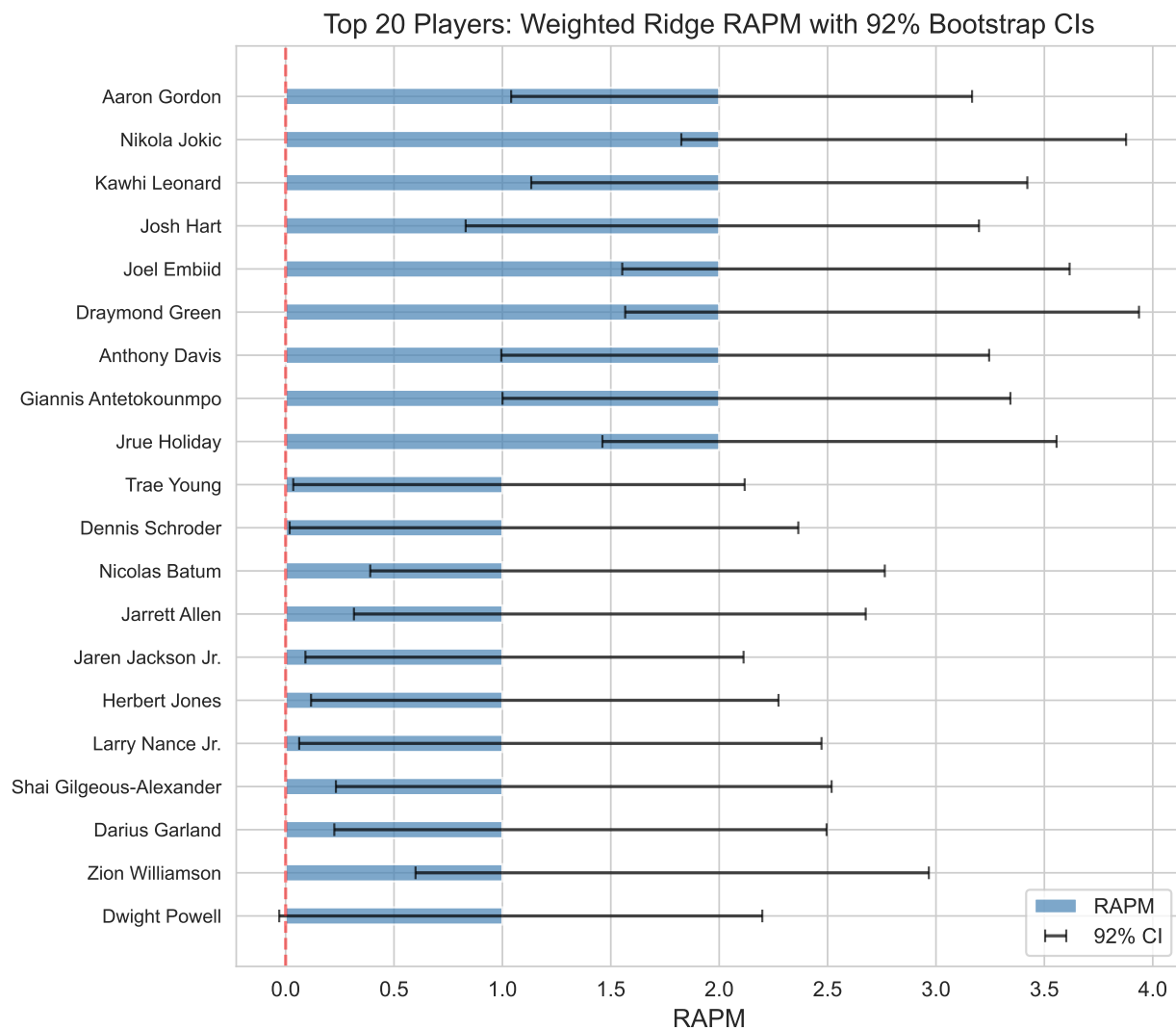


Figure 6: 92% Bootstrap Confidence Intervals for the Top 20 RAPM Players. The horizontal bars show the uncertainty range; overlapping intervals indicate that the precise ranking is uncertain.

Observations:

The confidence intervals for the top 20 players overlap a lot, which tells me the precise ordering is pretty uncertain. I can be fairly confident that the top-ranked players are above average (their intervals generally exclude zero), but I can't say for sure that the #1 player is truly better than the #10 player. That's just the nature of basketball data: even with a full season of stints, the signal from any single player is relatively weak compared to the total variance in scoring margins.

3.2 (b) Impact of Minutes Played on Confidence Intervals

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['boot_se'], alpha=0.5, s=20, color='coral')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('Bootstrap Standard Error', fontsize=12)
ax.set_title('Bootstrap Standard Error vs Minutes Played', fontsize=14)
plt.tight_layout()
plt.show()
```

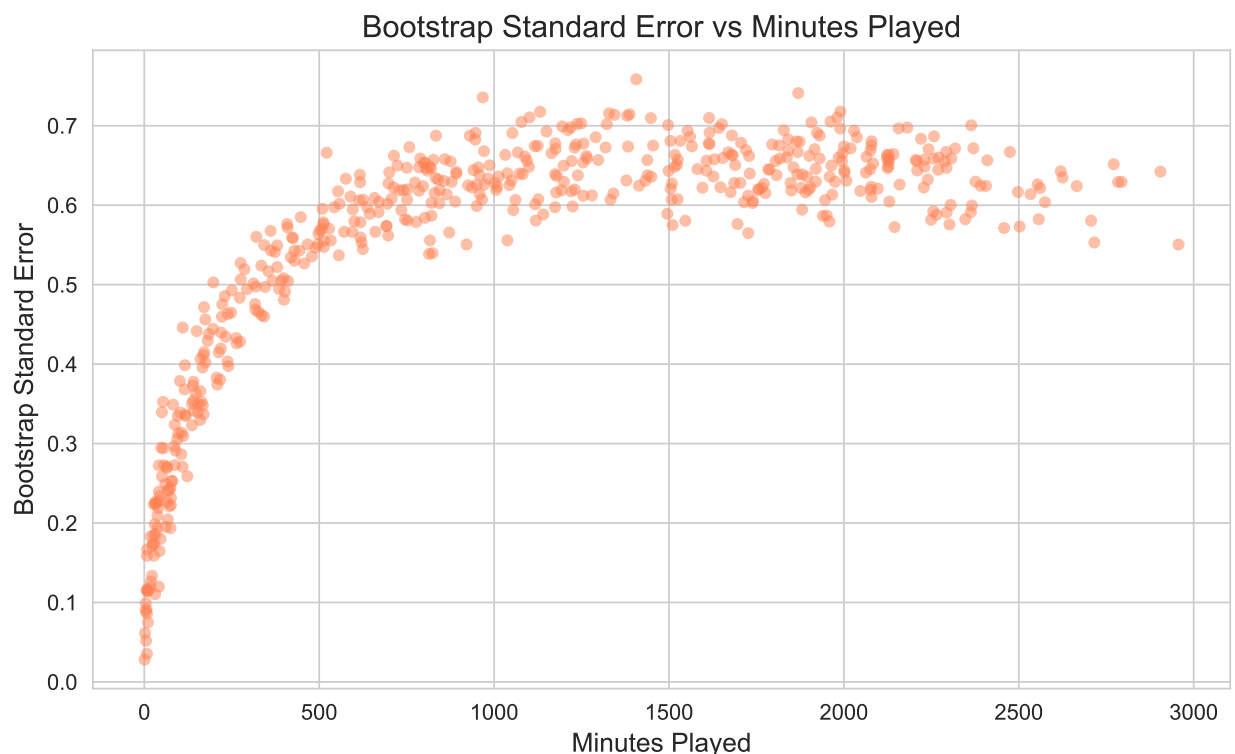


Figure 7: Bootstrap Standard Error vs Minutes Played. Low-minute players have artificially small bootstrap SEs due to ridge regularization pinning their coefficients near zero across resamples.

Observations:

There's a clear relationship between minutes played and bootstrap standard error. Players with very few minutes (under ~200) have noticeably smaller bootstrap SEs, which at first seems counterintuitive. This is actually a consequence of ridge regularization: for players with minimal data, the regularization penalty dominates every bootstrap resample, pinning their coefficient near zero regardless of which stints are drawn. The coefficient barely moves across iterations, so the bootstrap SE is artificially small. As minutes increase into the 500-1500 range, bootstrap SEs rise and roughly

plateau, because these players have enough data that resampling produces meaningful variation in their estimates.

This pattern is important to keep in mind:

- **Intuitively:** A player who was on the court for only 100 minutes across a handful of stints gives me very little data to estimate their true impact. We might expect high uncertainty, but the bootstrap reports low SE because ridge forces the estimate to stay near zero no matter what gets resampled. A player with 2,000+ minutes gives me way more observations, and the estimate genuinely varies across bootstrap samples because the data (not the penalty) is driving the coefficient.
- **Statistically:** In regression, a coefficient's standard error depends on how much variation exists in that feature. A player in many stints (high minutes) has more non-zero entries in their column of the stint matrix, giving more information for a precise estimate. Ridge regularization adds $\alpha \mathbf{I}$ to the precision matrix, which reduces the effective variance of all coefficients but hits low-information players the hardest. For high-minute players, the data contribution to the precision matrix dominates the regularization term, so their SEs behave roughly like the classical $1/\sqrt{n}$ scaling. For low-minute players, the regularization term dominates, which artificially stabilizes the bootstrap estimates and produces smaller SEs than a purely data-driven approach would. This is a known limitation of bootstrapping regularized estimators, and I revisit it in the Bayesian comparison below.

3.3 (c) Comparison to Bayesian Credible Intervals

```
# Bayesian standard errors using the provided formula
weights = WeightVector.values
stint_mat = StintMatrix.values
margin_vec = MarginVector.values

# Weighted residual variance
var = np.average(
    (margin_vec - model_ridge_w.predict(stint_mat))**2,
    weights=weights
)
num_players = stint_mat.shape[1]

# Posterior precision matrix:  $X^T W X + \alpha * I$ 
AMat = (stint_mat * weights.reshape(-1, 1)).T @ stint_mat + alpha_opt_w * np.eye(num_players)

# Posterior covariance
posterior_covariance = var * np.linalg.inv(AMat)
```

```

# Bayesian standard errors
se_bayesian = np.sqrt(np.diag(posterior_covariance))

# Add to dataframe
rapm_ridge_w['bayesian_se'] = se_bayesian

# Bayesian 92% credible intervals
from scipy.stats import norm
z_val = norm.ppf(0.96) # For 92% interval: 4% in each tail
rapm_ridge_w['bayes_CI_lower'] = rapm_ridge_w['RAPM'] - z_val * se_bayesian
rapm_ridge_w['bayes_CI_upper'] = rapm_ridge_w['RAPM'] + z_val * se_bayesian

fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Panel 1: Both SEs vs minutes played
axes[0].scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['boot_se'],
                alpha=0.4, s=15, color='coral', label='Bootstrap SE')
axes[0].scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['bayesian_se'],
                alpha=0.4, s=15, color='steelblue', label='Bayesian SE')
axes[0].set_xlabel('Minutes Played', fontsize=11)
axes[0].set_ylabel('Standard Error', fontsize=11)
axes[0].set_title('Standard Errors vs Minutes Played', fontsize=12)
axes[0].legend()

# Panel 2: Bootstrap SE vs Bayesian SE directly
axes[1].scatter(rapm_ridge_w['bayesian_se'], rapm_ridge_w['boot_se'],
                alpha=0.4, s=15, color='darkgreen')
max_se = max(rapm_ridge_w['bayesian_se'].max(), rapm_ridge_w['boot_se'].max())
axes[1].plot([0, max_se], [0, max_se], 'r--', alpha=0.5, label='y = x')
axes[1].set_xlabel('Bayesian SE', fontsize=11)
axes[1].set_ylabel('Bootstrap SE', fontsize=11)
axes[1].set_title('Bootstrap vs Bayesian Standard Errors', fontsize=12)
axes[1].legend()

plt.tight_layout()
plt.show()

```

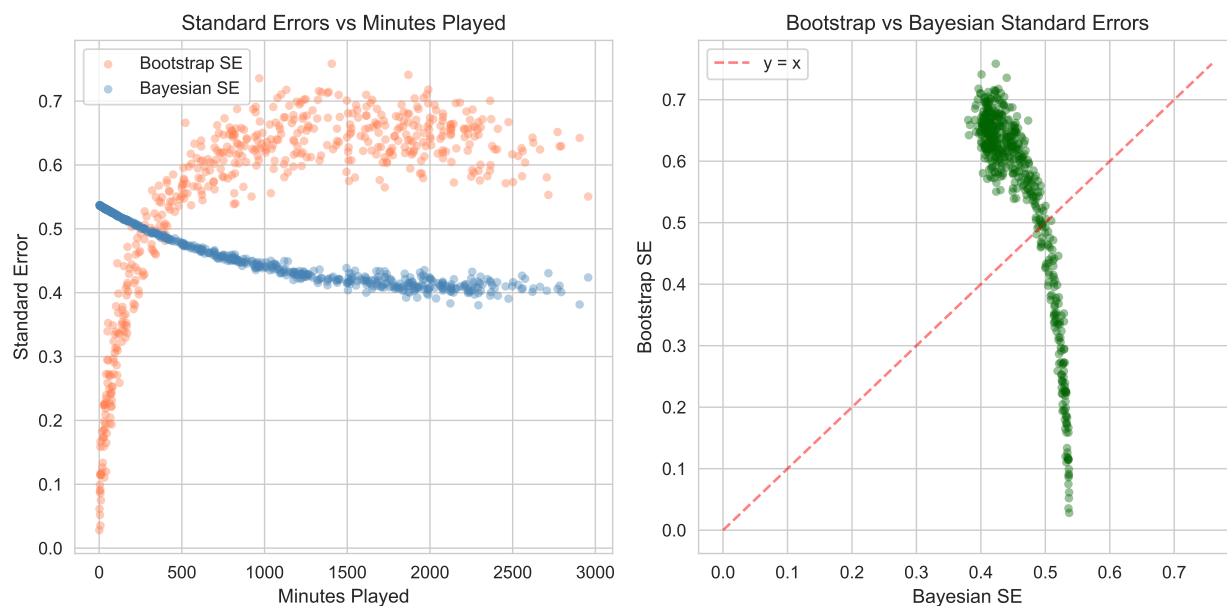


Figure 8: Comparison of Bayesian and Bootstrap Standard Errors vs Minutes Played.

```
# Quantify agreement by minutes-played bins
rapm_ridge_w['minutes_bin'] = pd.cut(rapm_ridge_w['minutes_played'],
                                     bins=[0, 200, 500, 1000, 2000, 4000],
                                     labels=['0-200', '200-500', '500-1000', '1000-2000', '2000-4000'])

se_comparison = rapm_ridge_w.groupby('minutes_bin', observed=True).agg(
    mean_boot_se=('boot_se', 'mean'),
    mean_bayes_se=('bayesian_se', 'mean'),
    n_players=('player_id', 'count')
).round(4)

se_comparison['ratio_boot_to_bayes'] = (se_comparison['mean_boot_se'] / se_comparison['mean_bayes_se'])
print("Mean Standard Errors by Minutes Played Bin:")
se_comparison
```

Mean Standard Errors by Minutes Played Bin:

minutes_bin	mean_boot_se	mean_bayes_se	n_players	ratio_boot_to_bayes
0-200	0.2617	0.5254	108	0.4980
200-500	0.4963	0.4933	62	1.0060
500-1000	0.6125	0.4571	105	1.3400

minutes_bin	mean_boot_se	mean_bayes_se	n_players	ratio_boot_to_bayes
1000-2000	0.6503	0.4216	180	1.5420
2000+	0.6364	0.4090	84	1.5560

Observations:

Comparing the Bayesian and bootstrap standard errors shows an interesting divergence that depends on minutes played:

- **Mid-range minutes (200-500, best agreement):** For players in this range, the Bayesian and bootstrap SEs are closest in magnitude (ratio very close to 1.0). Both methods produce similar uncertainty estimates because there’s enough data that the regularization penalty no longer dominates the bootstrap, but not so much that the two methods diverge for other reasons.
- **Low-minute players (biggest disagreement):** For players with very few minutes (0-200), the **bootstrap SEs are much smaller** than the Bayesian ones (ratio around 0.5). This seems surprising at first, but it follows directly from the bootstrapping-regularized-estimators issue discussed above: ridge regularization strongly shrinks low-minute player coefficients toward zero in *every* bootstrap sample. Since the estimate is always near zero no matter which stints get resampled, the coefficient barely moves across iterations, giving a small bootstrap SE. The Bayesian approach, on the other hand, correctly captures the large *posterior* uncertainty. Under the Bayesian interpretation of ridge, the prior is $\beta_j \sim N(0, \sigma^2/\alpha)$, and when the data contributes little information beyond the prior, the posterior remains broad. The Bayesian SE is saying: “I don’t know much about this player,” which is the right message.
- **High-minute players:** For high-minute players (2000+), the pattern flips: bootstrap SE exceeds Bayesian SE. This is because bootstrap resampling introduces real variability (different resamples give meaningfully different estimates), while the Bayesian posterior is tightly constrained by the large amount of data.

This divergence comes down to a fundamental difference in what each method measures:

- The **Bayesian credible interval** reflects posterior uncertainty *given the prior*. For low-minute players, the posterior stays broad because neither the prior nor the sparse data can nail down the coefficient. The Bayesian SE is basically saying: “I don’t know much about this player.”
- The **bootstrap confidence interval** measures the variability of the *estimator* across resampled datasets. For low-minute players, ridge regularization forces every bootstrap estimate near zero. The estimator is *stable* even though it’s *uninformative*. That’s a subtle but important distinction: low bootstrap variability doesn’t mean I actually know the true parameter well.

Which is more realistic? For low-minute players, the **Bayesian standard errors win hands down**. They correctly signal that I have very little information about these players. The bootstrap drastically underestimates uncertainty here because it confuses the stability of a heavily regularized estimator with genuine precision about a player's true ability. A GM relying on bootstrap CIs would be falsely confident about low-minute players, which could lead to bad roster decisions. For high-minute players, both approaches are reasonable, and the moderate difference matters less since both methods agree on the rough magnitude of uncertainty.

4 Extra Credit: Ridge vs Lasso

4.1 Ridge vs Lasso RAPM Comparison

The Lasso uses an L_1 penalty that drives many coefficients to exactly zero, essentially performing variable selection. This means the Lasso will effectively kick players out of the model entirely if their contribution isn't sufficiently supported by data.

```
# Define alpha grid for Lasso (different scale than Ridge)
alphas_lasso = np.logspace(-4, 2, 100)

# Fit LassoCV with weighted regression
model_lasso = LassoCV(alphas=alphas_lasso, fit_intercept=True, cv=5, max_iter=50000)
model_lasso.fit(StintMatrix.values, MarginVector.values, sample_weight=WeightVector.values)

alpha_opt_lasso = model_lasso.alpha_
print(f"Optimal Lasso alpha: {alpha_opt_lasso:.6f}")
print(f"Intercept: {model_lasso.intercept_:.4f}")
print(f"Number of non-zero coefficients: {np.sum(model_lasso.coef_ != 0)} out of {n_players}")
print(f"Number of zeroed-out players: {np.sum(model_lasso.coef_ == 0)}")
```

```
Optimal Lasso alpha: 0.026561
Intercept: 1.0788
Number of non-zero coefficients: 191 out of 539
Number of zeroed-out players: 348
```

```
# Build Lasso RAPM dataframe
rapm_lasso = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM_Lasso': model_lasso.coef_,
    'minutes_played': minutes_played
})
```

```

rapm_lasso = rapm_lasso.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Merge with Ridge for comparison
rapm_comparison = rapm_ridge_w[['player_id', 'player_name', 'RAPM', 'minutes_played']].rename(
    columns={'RAPM': 'RAPM_Ridge'})
).merge(
    rapm_lasso[['player_id', 'RAPM_Lasso']], on='player_id'
)

rapm_comparison['diff_ridge_minus_lasso'] = rapm_comparison['RAPM_Ridge'] - rapm_comparison['RAPM_Lasso']

# Top 20 by Lasso
rapm_lasso_sorted = rapm_comparison.sort_values('RAPM_Lasso', ascending=False).reset_index(drop=True)
print("Top 20 Players by Lasso RAPM:")
display_lasso = rapm_lasso_sorted[['player_name', 'RAPM_Lasso', 'RAPM_Ridge', 'minutes_played']]
for col in ['RAPM_Lasso', 'RAPM_Ridge']:
    display_lasso[col] = display_lasso[col].map('{:.3f}'.format)
display_lasso['minutes_played'] = display_lasso['minutes_played'].map('{:.1f}'.format)
display_lasso

```

Top 20 Players by Lasso RAPM:

	player_name	RAPM_Lasso	RAPM_Ridge	minutes_played
0	Joel Embiid	5.860	2.000	2249.5
1	Draymond Green	5.853	2.000	2019.4
2	Nikola Jokic	5.562	2.000	2255.5
3	Kawhi Leonard	4.384	2.000	1817.2
4	Anthony Davis	4.197	2.000	1880.8
5	Christian Koloko	4.061	1.000	757.8
6	Jrue Holiday	3.945	2.000	2097.0
7	Zion Williamson	3.827	1.000	859.1
8	Giannis Antetokounmpo	3.475	2.000	1840.6
9	Cameron Johnson	3.308	1.000	1073.0
10	Josh Hart	3.175	2.000	2474.7
11	Derrick White	3.086	1.000	2078.7
12	Julius Randle	2.980	1.000	2956.6
13	Jayson Tatum	2.932	1.000	2771.4
14	Lauri Markkanen	2.597	1.000	2129.9
15	Desmond Bane	2.594	1.000	1706.0
16	Shai Gilgeous-Alexander	2.569	1.000	2620.0

	player_name	RAPM_Lasso	RAPM_Ridge	minutes_played
17	Darius Garland	2.561	1.000	2209.8
18	Aaron Gordon	2.545	2.000	1966.7
19	Jarrett Allen	2.497	1.000	2076.5

```

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(rapm_comparison['RAPM_Ridge'], rapm_comparison['RAPM_Lasso'],
           alpha=0.4, s=15, c=rapm_comparison['minutes_played'],
           cmap='viridis', edgecolors='none')
cbar = plt.colorbar(ax.collections[0], ax=ax, label='Minutes Played')

max_val = max(abs(rapm_comparison['RAPM_Ridge']).max(), abs(rapm_comparison['RAPM_Lasso']).max())
ax.plot([-max_val, max_val], [-max_val, max_val], 'r--', alpha=0.5, label='y = x')
ax.set_xlabel('Ridge RAPM', fontsize=12)
ax.set_ylabel('Lasso RAPM', fontsize=12)
ax.set_title('Ridge vs Lasso RAPM Coefficients', fontsize=14)
ax.legend()
ax.axhline(0, color='gray', alpha=0.3)
ax.axvline(0, color='gray', alpha=0.3)
plt.tight_layout()
plt.show()

```

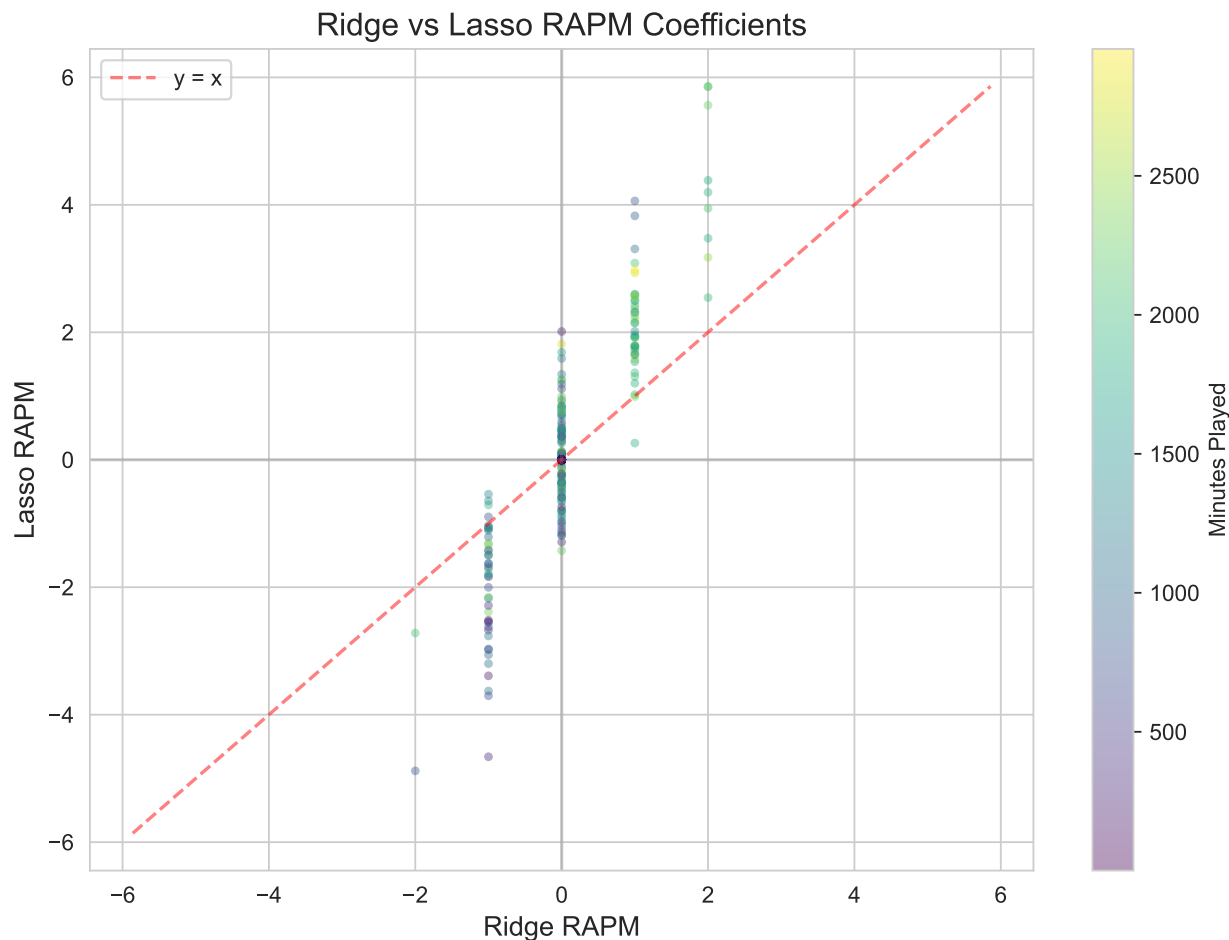


Figure 9: Ridge RAPM vs Lasso RAPM. Most players are along the diagonal, but the Lasso zeroes out many low-information players.

```
# Top 10 where Ridge >> Lasso
ridge_favored = rapm_comparison.nlargest(10, 'diff_ridge_minus_lasso')
print("Top 10 Players: Ridge RAPM >> Lasso RAPM")
print("(Ridge rates these players higher than Lasso)")
display_rf = ridge_favored[['player_name', 'RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso']]
for col in ['RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso']:
    display_rf[col] = display_rf[col].map('{:.3f}'.format)
display_rf['minutes_played'] = display_rf['minutes_played'].map('{:.1f}'.format)
display_rf
```

Top 10 Players: Ridge RAPM >> Lasso RAPM
(Ridge rates these players higher than Lasso)

	player_name	RAPM_Ridge	RAPM_Lasso	diff_ridge_minus_lasso	minutes_played
292	McKinley Wright IV	-1.000	-4.660	3.660	335.5
67	Simone Fontecchio	-2.000	-4.881	2.881	745.4
331	Aleksej Pokusevski	-1.000	-3.702	2.702	700.2
168	Lonnie Walker IV	-1.000	-3.628	2.628	1199.5
255	Thanasis Antetokounmpo	-1.000	-3.391	2.391	208.7
263	Jeff Green	-1.000	-3.199	2.199	1067.7
213	Will Barton	-1.000	-3.060	2.060	955.0
383	Blake Wesley	-1.000	-2.978	1.978	670.9
142	Devin Vassell	-1.000	-2.971	1.971	1116.1
249	Jordan Nwora	-1.000	-2.765	1.765	1174.4

```
# Top 10 where Lasso >> Ridge
lasso_favored = rapm_comparison.nsmallest(10, 'diff_ridge_minus_lasso')
print("\nTop 10 Players: Lasso RAPM >> Ridge RAPM")
print("(Lasso rates these players higher than Ridge)")
display_lf = lasso_favored[['player_name', 'RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso']]
for col in ['RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso']:
    display_lf[col] = display_lf[col].map('{:.3f}'.format)
display_lf['minutes_played'] = display_lf['minutes_played'].map('{:.1f}'.format)
display_lf
```

Top 10 Players: Lasso RAPM >> Ridge RAPM
(Lasso rates these players higher than Ridge)

	player_name	RAPM_Ridge	RAPM_Lasso	diff_ridge_minus_lasso	minutes_played
156	Joel Embiid	2.000	5.860	-3.860	2249.5
2	Draymond Green	2.000	5.853	-3.853	2019.4
258	Nikola Jokic	2.000	5.562	-3.562	2255.5
105	Christian Koloko	1.000	4.061	-3.061	757.8
419	Zion Williamson	1.000	3.827	-2.827	859.1
306	Kawhi Leonard	2.000	4.384	-2.384	1817.2
26	Cameron Johnson	1.000	3.308	-2.308	1073.0
167	Anthony Davis	2.000	4.197	-2.197	1880.8
16	Derrick White	1.000	3.086	-2.086	2078.7
344	Boban Marjanovic	0.000	2.013	-2.013	166.6

```

ridge_top20 = rapm_comparison.nlargest(20, 'RAPM_Ridge')
lasso_top20 = rapm_comparison[rapm_comparison['RAPM_Lasso'] != 0].nlargest(20, 'RAPM_Lasso')

fig, axes = plt.subplots(1, 2, figsize=(10, 7))

# Ridge top 20
r20 = ridge_top20.iloc[:, -1]
axes[0].barh(range(20), r20['RAPM_Ridge'], color='steelblue', alpha=0.7)
axes[0].set_yticks(range(20))
axes[0].set_yticklabels(r20['player_name'], fontsize=8)
axes[0].set_xlabel('RAPM')
axes[0].set_title('Top 20: Ridge RAPM')

# Lasso top 20
l20 = lasso_top20.iloc[:, -1]
axes[1].barh(range(len(l20)), l20['RAPM_Lasso'], color='darkgreen', alpha=0.7)
axes[1].set_yticks(range(len(l20)))
axes[1].set_yticklabels(l20['player_name'], fontsize=8)
axes[1].set_xlabel('RAPM')
axes[1].set_title('Top 20: Lasso RAPM')

plt.tight_layout()
plt.show()

```

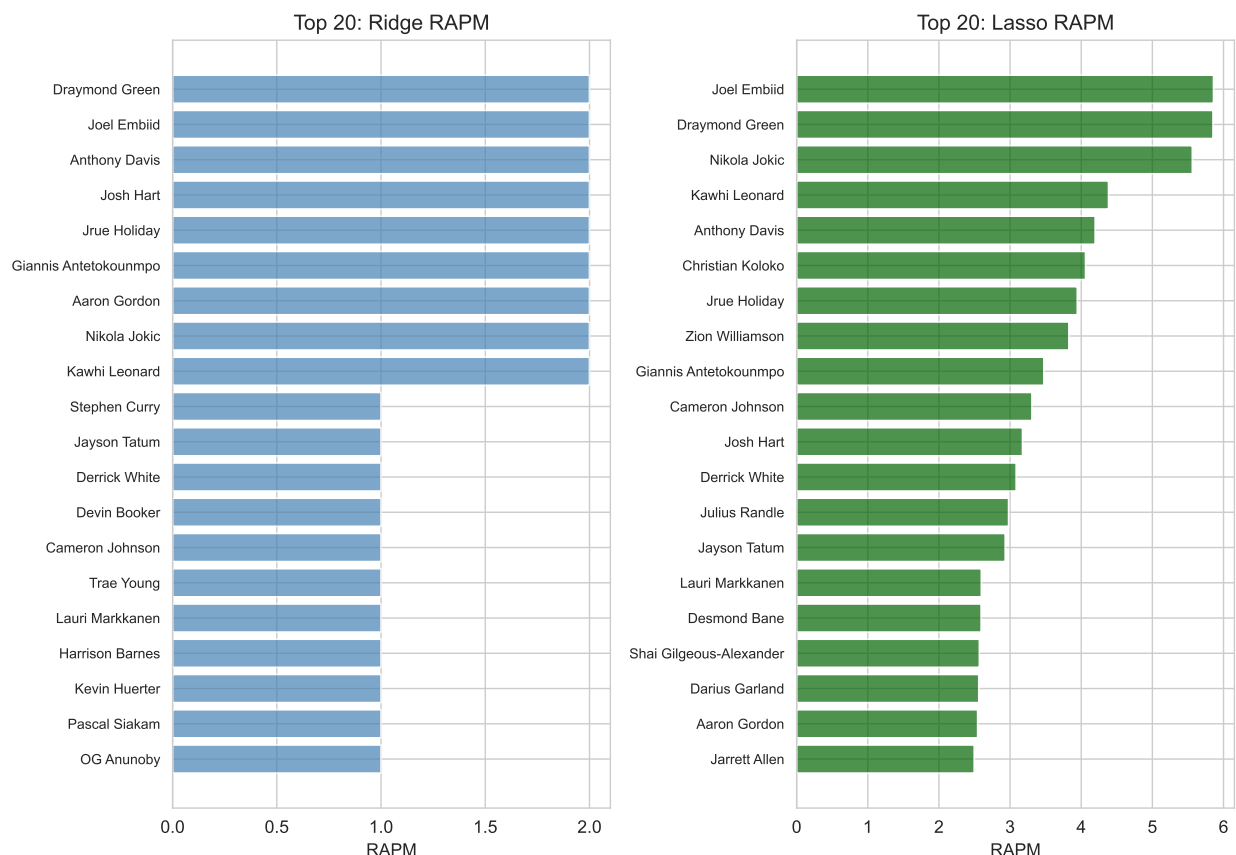


Figure 10: Side-by-side comparison of Top 20 players ranked by Ridge and Lasso RAPM.

Discussion:

The Lasso and Ridge give meaningfully different RAPM rankings, which reflects their different philosophies:

1. **Sparsity:** The Lasso sets many player coefficients to exactly zero, basically saying “there’s not enough evidence that this player differs from average.” Ridge never fully zeroes out a coefficient; it shrinks toward zero but always keeps some non-zero estimate. So Lasso is more aggressive with variable selection.
2. **Top 20 differences:** The Lasso tends to give more extreme non-zero coefficients for the players it keeps. This happens because Ridge shrinks *all* coefficients proportionally, spreading the penalty across every player, while the Lasso zeroes out a bunch of players and concentrates the remaining coefficient “budget” on the survivors. Those survivors end up less penalized overall, so their RAPM magnitudes can be larger than Ridge’s.
3. **Disagreements:** The biggest disagreements show up for players with moderate playing time, enough for Ridge to assign them a meaningful coefficient, but not enough for Lasso to consider the signal strong enough to survive. Looking at how these players actually performed in the 2022-23 season could help assess which model was more predictive:

- Players where **Ridge rated higher than Lasso** (e.g., McKinley Wright IV, Simone Fontecchio, Aleksej Pokusevski) tend to be lower-minute players with negative RAPM. Ridge assigns them small negative coefficients while the Lasso amplifies their negative signal because the L_1 penalty, once it decides to keep a coefficient, does not shrink it as aggressively as Ridge does. In other words, Ridge spreads the penalty evenly and keeps these players closer to zero, while the Lasso either zeros a player out or lets a larger magnitude through.
- Players where **Lasso rated higher than Ridge** (e.g., Joel Embiid, Draymond Green, Nikola Jokic) are high-minute stars with strong positive signal. The Lasso gives them larger coefficients because, having zeroed out ~350 players, the remaining penalty budget is spread across fewer active coefficients. Ridge, by contrast, penalizes all 539 players equally, which drags even the best players' estimates down somewhat.

Overall, I think **Ridge regression is the better default choice for RAPM** because the underlying assumption, that all players have *some* non-zero impact on the game, is more natural for basketball. While the Lasso's sparsity assumption has appeal (some end-of-bench players may genuinely be indistinguishable from replacement level), it can also discard moderate-impact role players who do contribute meaningfully. That said, Lasso is useful as a complementary tool: the players whose signals survive the L_1 penalty are the ones about whom I can be most confident, and a front office might use the Lasso's non-zero set as a shortlist of clearly impactful players worth further evaluation.